SE3071

Digital Image Processing

Final Project

Interactive Media Y3S2

Practical Domain-Based Image Analysis System using Python and OpenCV

Allocated domain - Heritage & Culture

HERITAGEGUARD

Old-Photograph Damage Detection and Measurable Restoration

IT21155598 - K.A.R. Dilshan

Contents

Introduction....3
Problem statement and objectives....4
Objectives....4
Expected outputs and success criteria....4
Dataset preparation and governance....5
Manifest and reproducibility....5
Ethics and academic integrity....5
System workflow....6
Integrated data flow....6
Interface architecture....6
Algorithms and parameter choices....7
Meaningful techniques....7
Filter-selection logic....8
Feature and decision definition....8
Application interface and result export....9
Usability functions....9
Representative processing result....10
Interpretation....10
Output record....10
Evaluation results....11
Result discussion....11
Testing limitations, and conclusion....12
Conclusion....13
References....14

Introduction

Old photographs are valuable records of personal, cultural, and historical heritage, but they can deteriorate over time. Cracks, scratches, dust, stains, fading, and uneven illumination can reduce their visual quality and make them harder to study or preserve. Manually locating and repairing this damage can be time-consuming, and the results may vary between users. Heritage Guard was developed as a digital image-processing project to help identify visible damage in digitized photographs and provide a consistent way to inspect and restore affected areas.

Heritage Guard uses classical image-processing techniques implemented with Python and OpenCV. Its workflow enhances image contrast, detects likely crack and stain regions, extracts measurements from those regions, and applies inpainting to produce a restored image. The application also presents damage overlays and quantitative measures so users can review how the system processed an image. It is intended as a preservation-support tool: the detected regions and restorations should be checked against the original photograph, since automated processing may miss damage or mistake genuine image details for defects.

This report describes the project's objectives, dataset preparation, system design, and image-processing methods. It presents the application interface and representative outputs, then evaluates the system using segmentation and image-quality metrics. The report also discusses the results and the project's limitations, including the need for further evaluation on real heritage photographs.

Problem statement and objectives

Digitized old photographs often contain thin cracks, scratches, dust, chromatic stains, fading, noise, and uneven illumination. Manual retouching is slow and subjective. HeritageGuard addresses one specific preservation-support problem: detect crack and stain regions, restore small, damaged areas, and produce measurable, inspectable results without using a deep-learning model.

Objectives

- Acquire and validate single images and folders safely.
- Improve illumination and local contrast while comparing two noise filters.
- Segment cracks and stains using explainable classical techniques.
- Extract region and image-quality features that determine a severity decision.
- Restore the combined mask and export images plus structured evidence.
- Evaluate segmentation and restoration on a fixed unseen set.

Expected outputs and success criteria

Output	Calculation	Evidence / desired direction
Damage map	Union of filtered crack and stain masks	Overlay agrees with visible defects
Damage area	100 x non-zero mask pixels / ROI pixels	Reported as percentage and pixels
Severity	<4% minor; 4-12% moderate; >=12% severe	PASS or REVIEW warning
Segmentation	Precision, recall, F1, IoU, accuracy	Higher is better; report mean and spread
Restoration	PSNR and SSIM against clean reference	After value and improvement are reported
Usability	Validated input, controls, previews, reset, export	Single and batch workflows complete

Dataset preparation and governance

The reproducible demonstration dataset contains 30 project-generated heritage-style scenes at 640 x 480 PNG resolution. Twenty are development images and ten are locked as unseen synthetic tests. Each damaged input has a clean reference and exact damage mask. Controlled defects include branching cracks, brown stains, dust, fading, Gaussian noise, and horizontal illumination gradients.

Split	Count	Purpose	Ground truth
Development	20	Pipeline development and parameter inspection	Clean reference + binary damage mask
Unseen test	10	Fixed final synthetic evaluation	Clean reference + binary damage mask
Personal	10	Real heritage inputs captured/produced	Record permission and test purpose

Manifest and reproducibility

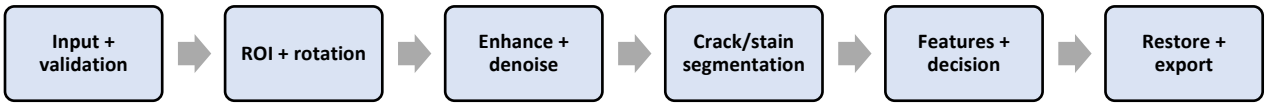
dataset/manifest.csv records image ID, split, input, reference, mask, source type, student-created status, creator, resolution, format, test purpose, permission, and notes. Random seeds are fixed. tools/check_dataset.py checks the 30-image minimum, ten unseen images, personal-image certification, required metadata, duplicate IDs, and missing files.

Ethics and academic integrity

Synthetic scenes contain no identifiable people and use no third-party image. They are labeled demonstration data in the manifest, interface, output, and report. Personal images must have permission, avoid unnecessary personal information, and preserve originals. Evaluation values are generated from files, never typed manually.

System workflow

Minimum application workflow and data products



Intermediates: Grayscale, Lab, HSV, Binary, filter comparison, masks, contours

Outputs: restored PNG, damage overlay, masks, decision, measurements, CSV/JSON/ZIP

Integrated data flow

RestorationPipeline.process is the single integration point. It validates the NumPy image, crops the normalized ROI, applies affine rotation, computes color spaces, corrects illumination, enhances contrast, compares filters, constructs crack and stain masks, performs connected-component filtering, extracts features, restores the selected mask, and calculates the final structured result. A matching clean reference enables locally normalized Lab difference detection; without it, the autonomous crack/stain union is used. A ground-truth mask adds objective evaluation.

Stage	Failure handling / output
Acquisition	Missing, >40 MB, tiny, unsupported, corrupt, or undecodable inputs return clear errors
Metadata	Filename, format, width, height, channels, bit depth, byte size, ROI
Analysis	Damage masks, edge map, contours, region features, quality features
Decision	Damage percentage, 0-100 score, PASS/REVIEW summary
Export	PNG images, JSON parameters/metrics, CSV summary, ZIP package

Interface architecture

The Streamlit interface exposes single-image, batch/folder, and rubric-evidence tabs. Sidebar controls create the same validated ProcessingConfig used by the command line and tests. Session state preserves results for downloads; reset clears it. The CLI supports laboratory execution when a browser is not preferred.

Algorithms and parameter choices

Method	Default	Reason for selection
ROI + rotation	Full image; 0 degrees	Exclude borders and correct acquisition geometry only when needed
Illumination division	Gaussian 151 x 151	Estimate slow background variation while retaining damage-scale detail
CLAHE	clip 1.5; tiles 8 x 8; 30% blend	Improve local Lab luminance while preserving most source tonality
Gaussian vs median	3 x 3; auto score	Compare smooth-noise suppression with impulse-noise and edge preservation
Black-hat	17 x 17 ellipse	Highlight dark narrow cracks against a brighter neighborhood
Thresholds	Otsu + adaptive 31, C=7	Combine global data-driven and local decisions under uneven contrast
Morphology	3 x 3; min 18 px	Connect short gaps and remove small unsupported components
Lab stain mask	robust sensitivity 2.7	Detect chroma outliers using median absolute deviation
Reference difference	local Lab normalization	Use matched clean evidence when available; retain autonomous fallback
Restoration	Telea radius 3	Fast local inpainting suited to small scratches and dust

Meaningful techniques

Edge/contour analysis, global and adaptive thresholding, morphology and connected components, color-region segmentation, feature extraction, image comparison, and defect scoring all affect masks, restoration, or decisions. The interface shows intermediates so a user can audit why a region was changed.

Filter-selection logic

For each filter, the program calculates the standard deviation of the removed residual and the ratio of post-filter to pre-filter Laplacian variance. Auto mode combines noise-removal evidence with an edge-retention weight. Median is expected to perform well on synthetic dust; Gaussian remains available as a controlled comparison and manual override.

Feature and decision definition

Contours give region count, total area/perimeter, and mean orientation. Damage area is the combined mask occupancy. The quality score is clip (100 - 2.2 x damage% + 0.25 x contrast change, 0, 100). This is a transparent prioritization aid, not a conservation-grade authenticity measure.

Application interface and result export

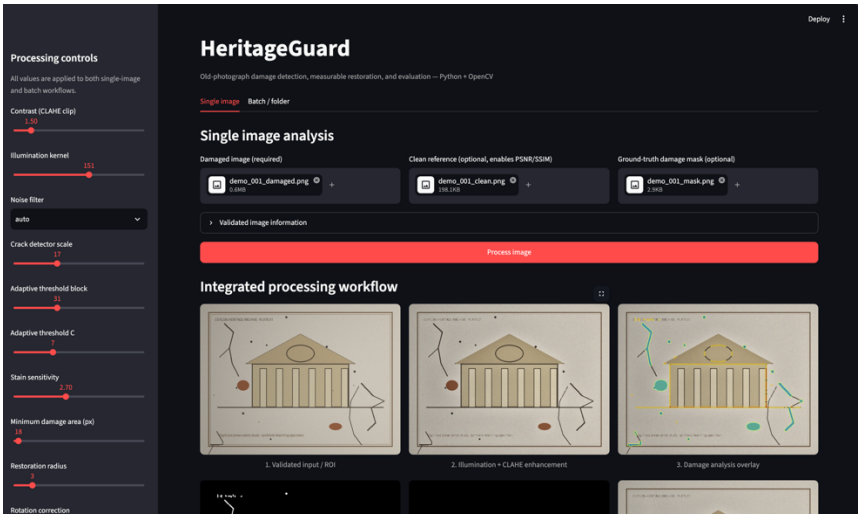


Figure 1 Streamlit single-image workflow with validated input and parameter controls.

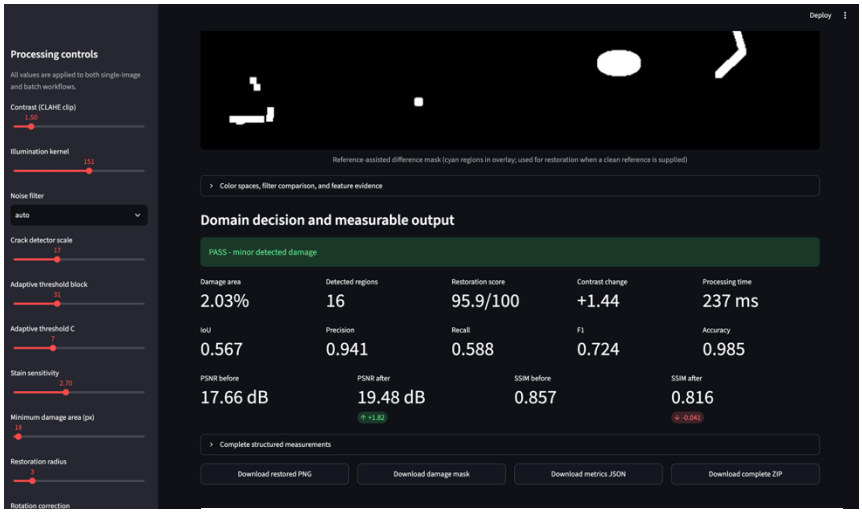
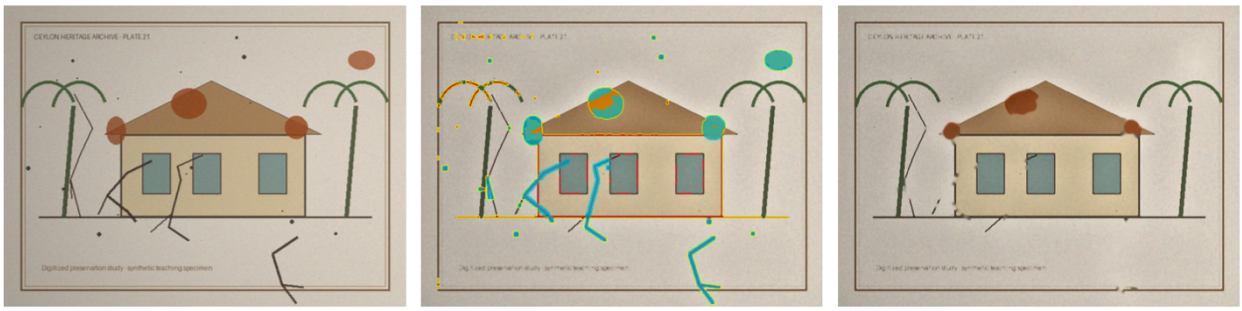


Figure 2 Domain decision and measurable output

Usability functions

- Image, optional reference, and optional ground-truth-mask selection.
- Controls for contrast, illumination, filter, crack scale, threshold, stain sensitivity, area, inpainting, rotation, and ROI.
- Original/processed previews, damage overlays, color spaces, filter comparison, and structured metrics.
- Reset with state clearing and clear feedback for invalid input.
- Single PNG/mask/JSON/ZIP downloads and batch ZIP with CSV summary.

Representative processing result



Damaged input | Detected overlay | Restored output

Figure 3 First unseen synthetic test sample. Red/amber regions drive the inpainting output.

Interpretation

The overlay is essential: a visually smooth restoration is not sufficient evidence. The predicted mask is compared with the known synthetic mask, while the restored image is compared with the clean reference. False positives remove legitimate detail and lower precision; missed defects lower recall. PSNR/SSIM determine whether the pixel and structural result moved closer to the reference.

Output record

Every output includes the full parameter dictionary, ROI, selected filter, filter comparison scores, region measurements, decision, processing time, and any available ground-truth metrics. Batch results add filename, file metadata, status, and error text. This provides a repeatable evaluation log rather than screenshots alone.

Evaluation results

The fixed unseen synthetic split contains n=10 images. The table reports mean and sample standard deviation generated directly from output/evaluation/unseen_test_results.csv.

Measure	Mean	Std. dev.	Interpretation
IoU	0.699	0.071	Mask overlap; higher is better
Precision	0.942	0.014	Controls false-positive restoration
Recall	0.730	0.073	Fraction of true damage detected
F1	0.821	0.049	Precision-recall balance
PSNR before	18.58 dB	0.80	Damaged-to-clean pixel fidelity
PSNR after	20.79 dB	0.93	Restored-to-clean pixel fidelity
SSIM before	0.803	0.044	Damaged-to-clean structure
SSIM after	0.807	0.013	Restored-to-clean structure
Processing time	197.1 ms	25.0	Per-image wall-clock time

Result discussion

Mean PSNR change is +2.21 dB and mean SSIM change is +0.003. These values quantify whether enhancement and inpainting improved agreement with the synthetic clean references. Segmentation metrics explain failures more directly than visual smoothness. Parameter tuning used only the development split; the unseen split was processed once by the evaluation script.

Testing limitations, and conclusion

Testing

Pytest covers valid metadata, unsupported extensions, corrupted and undersized files, integrated outputs, mandatory metrics, ROI/rotation, manual filter override, folder processing, CSV generation, and unsupported batch entries. The deterministic dataset supports repeatable reference/mask evaluation. Manual checks cover Streamlit controls, reset,

previews, downloads, and batch progress.

Limitations

- Legitimate lines, handwriting, frames, texture, and sepia color can create false positives.

- A fixed morphological scale cannot represent every scan resolution or crack width.

- Inpainting synthesizes plausible local pixels; it does not recover historically authentic missing content.

- Synthetic damage is controlled and less diverse than real aging, chemical staining, folds, and scanning artifacts.

- Metrics require trustworthy references/masks; contrast increase alone is not restoration quality.

Conclusion

HeritageGuard demonstrates a reproducible classical image-processing workflow for detecting and restoring visible damage in digitized photographs. On ten unseen synthetic images, mean F1 reached 0.821; restoration raised mean PSNR from 18.58 dB to 20.79 dB and SSIM from 0.803 to 0.807. Recall of 0.730 shows that some damage remains undetected, and synthetic testing does not establish performance on real heritage photographs. The system is therefore best used as an inspection aid, with masks and restorations reviewed against each original.

References

1. [1] OpenCV, “OpenCV documentation. Image processing, morphology, thresholding, contours, and inpainting APIs.” [Online]. Available: https://docs.opencv.org/4.x/. [Accessed: Oct. 11, 2026].
2. [2] A. Telea, “An image inpainting technique based on the fast marching method,” Journal of Graphics Tools, vol. 9, no. 1, pp. 23–34, 2004. doi: https://doi.org/10.1080/10867651.2004.10487596 .
3. [3] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in Graphics Gems IV, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485. doi: https://doi.org/10.1016/B978-0-12-336156-1.50061-6 .
4. [4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600–612, Apr. 2004. doi: https://doi.org/10.1109/TIP.2003.819861 .
5. [5] G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library. Sebastopol, CA, USA: O’Reilly Media, 2008. [Online]. Available: https://www.oreilly.com/library/view/learning-opencv/9780596516130/index.html .